

TPI

TRABAJO PRÁCTICO INTEGRADOR

PROGRAMACIÓN EN PYTHON

Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas

Alumno/a: Valentino Altamira, Angel sanchez (grupo 99)

Coomision: COM6, COM4

Fecha: 11/6/2026

Institución: Universidad Tecnologica Nacional

Indice

Introducción

Marco Teórico

Decisiones Técnicas y Arquitectura

Capturas de Ejecución

Dificultades Encontradas

Conclusiones

Bibliografía / Webgrafía

Introduccion

En el presente Trabajo Práctico Integrador se desarrolló una aplicación en Python orientada a la gestión de información de distintos países. El objetivo principal fue aplicar los conocimientos adquiridos durante la cursada de Programación 1, utilizando estructuras de datos como listas y diccionarios, además de funciones, condicionales, ciclos repetitivos y manejo de archivos CSV.

El sistema permite realizar diversas operaciones sobre un conjunto de países, tales como agregar nuevos registros, actualizar información existente, realizar búsquedas, aplicar filtros, ordenar datos y obtener estadísticas relevantes. Para ello se trabajó con información almacenada en un archivo CSV, que es procesada y administrada por el programa mediante un menú interactivo en consola.

A lo largo del desarrollo se pusieron en práctica conceptos fundamentales de programación, prestando especial atención a la organización del código, la modularización mediante funciones y la validación de datos ingresados por el usuario. Este proyecto permitió afianzar habilidades relacionadas con la resolución de problemas, el análisis lógico y la implementación de soluciones informáticas utilizando el lenguaje Python.

Finalmente, el trabajo representa una integración de los contenidos estudiados durante la materia, demostrando la capacidad de desarrollar una aplicación funcional que cumpla con los requerimientos planteados en la consigna.

Marco teorico

Listas

Las listas son estructuras de datos que permiten almacenar múltiples elementos en una sola variable. En Python, las listas son ordenadas y pueden modificarse durante la ejecución del programa. En este proyecto se utilizaron para almacenar el conjunto de países cargados desde el archivo CSV, permitiendo recorrerlos, agregarlos y modificarlos según las necesidades del sistema.

Diccionarios

Los diccionarios son estructuras que almacenan información mediante pares clave-valor. Cada país del sistema fue representado mediante un diccionario que contiene datos como nombre, continente, población y superficie. Esta estructura facilita el acceso y la manipulación de la información de forma organizada.

Funciones

Las funciones permiten dividir un programa en bloques de código reutilizables que realizan tareas específicas. Su utilización mejora la organización, legibilidad y mantenimiento del código. En este proyecto se implementaron funciones para agregar países, actualizar datos, realizar búsquedas, aplicar filtros, ordenar registros y generar estadísticas.

Estructuras Condicionales

Las estructuras condicionales permiten ejecutar diferentes acciones dependiendo de si una condición es verdadera o falsa. En Python se implementan principalmente mediante las sentencias `if`, `elif` y `else`. Estas estructuras fueron utilizadas para controlar las opciones del menú y validar distintas situaciones durante la ejecución del programa.

Estructuras Repetitivas

Las estructuras repetitivas permiten ejecutar un bloque de instrucciones varias veces. En este trabajo se utilizaron bucles `for` para recorrer listas de países y `while` para mantener activo el menú principal hasta que el usuario decida finalizar el programa.

Archivos CSV

CSV (Comma Separated Values) es un formato de archivo utilizado para almacenar datos tabulares de manera sencilla. Cada línea representa un registro y cada valor se encuentra separado por comas. En este proyecto se utilizó un archivo CSV para almacenar la información inicial de los países y cargarla al programa mediante la biblioteca `csv` de Python.

Ordenamiento de Datos

El ordenamiento consiste en organizar los datos según un criterio determinado. Python ofrece funciones como `sorted()`, que permiten ordenar listas utilizando diferentes campos. En este sistema se implementó el ordenamiento de países por nombre, población y superficie para facilitar la consulta de la información.

Estadísticas Básicas

Las estadísticas básicas permiten obtener información relevante a partir de un conjunto de datos. Entre las operaciones implementadas en este proyecto se encuentran el cálculo del país con mayor y menor población, el promedio de población, el promedio de superficie y la cantidad de países por continente. Estas métricas permiten realizar un análisis general de la información almacenada.

Validación de Datos

La validación de datos consiste en verificar que la información ingresada por el usuario sea correcta antes de ser procesada por el sistema. En este trabajo se implementaron controles básicos para evitar errores al ingresar valores numéricos incorrectos y para manejar situaciones en las que no se encuentren resultados durante las búsquedas o filtros.

Modularización

La modularización es una técnica de programación que consiste en dividir un programa en partes más pequeñas e independientes. Cada módulo o función cumple una responsabilidad específica, lo que facilita la comprensión, el mantenimiento y la reutilización del código. Este enfoque fue aplicado durante el desarrollo del sistema mediante el uso de funciones para cada una de las operaciones principales.

Decisiones técnicas y arquitectura

Funciones Utilizadas

Durante el desarrollo del sistema se implementaron diversas funciones con el objetivo de modularizar el código y asignar una responsabilidad específica a cada operación.

`agregar_pais()`

Permite ingresar un nuevo país al sistema solicitando nombre, continente, población y superficie. Además, realiza validaciones básicas para evitar errores en el ingreso de datos.

`actualizar_pais()`

Permite modificar la población y la superficie de un país existente. Para ello, busca el país por nombre y actualiza los valores ingresados por el usuario.

`buscar_pais()`

Realiza la búsqueda de un país por nombre, permitiendo coincidencias parciales. En caso de encontrar resultados, muestra la información correspondiente; de lo contrario, informa que no se encontraron coincidencias.

`filtrar_paises()`

Permite filtrar países según diferentes criterios, como continente, rango de población y rango de superficie. Esta funcionalidad facilita la consulta específica de datos dentro del conjunto almacenado.

`ordenar_paises()`

Ordena la información de los países según distintos criterios seleccionados por el usuario, como nombre, continente, población o superficie.

`mostrar_estadisticas()`

Calcula y muestra estadísticas generales del conjunto de datos, incluyendo país con mayor y menor población, promedio de población, promedio de superficie y cantidad de países por continente.

`mostrar_todos()`

Muestra en pantalla todos los países almacenados en el sistema junto con sus datos correspondientes.

Beneficios del uso de funciones

La utilización de funciones permitió dividir el programa en módulos independientes, mejorando la organización, legibilidad y mantenimiento del código. Además, este enfoque facilita la reutilización de procedimientos y reduce la repetición innecesaria de instrucciones.

CAPTURAS DEL CODIGO EN PHYTON


```
File Edit Selection View Go Run ... Search
import csv.py X
C:\Users\valen_5p466wg> Downloads> tpi prog > import csv.py > ...
1 import csv
2
3 paises = []
4
5 with open("paises.csv", encoding="utf-8") as archivo:
6     lector = csv.DictReader(archivo)
7
8     for fila in lector:
9         paises.append({
10             "nombre": fila["nombre"],
11             "continente": fila["continente"],
12             "poblacion": int(fila["poblacion"]),
13             "superficie": int(fila["superficie"])
14         })
15
16 def agregar_pais(paises):
17     try:
18         nombre = input("Nombre del país: ").strip()
19         continente = input("Continente: ").strip()
20         poblacion = int(input("Población: ").strip())
21         superficie = int(input("Superficie: ").strip())
22
23         paises.append({
24             "nombre": nombre,
25             "continente": continente,
26             "poblacion": poblacion,
27             "superficie": superficie
28         })
29         print("País agregado correctamente.")
30     except ValueError:
31         print("Error: Ingrese valores numéricos para población y superficie.")
32
33 def actualizar_pais(paises):
34     nombre_buscar = input("Ingrese país a actualizar: ")
35
36     encontrado = False
37
38     for pais in paises:
39         if pais["nombre"].lower() == nombre_buscar.lower():
40             try:
41                 pais["poblacion"] = int(input("Nueva población: "))
42                 pais["superficie"] = int(input("Nueva superficie: "))
43
44                 print("Datos actualizados")
45             except ValueError:
46                 print("Error: Ingrese valores numéricos para población y superficie")
47
48
49
50
Python 3.14 (64-bit)
```

```
File Edit Selection View Go Run ... < -> Search
import csv.py X
C:\Users\valen_5p466wg>Downloads>tpi prog > import csv.py > ...
33 def actualizar_pais(países):
50
51     encontrado = True
52     break
53
54     if not encontrado:
55         print("País no encontrado")
56
57 def buscar_pais(países):
58     texto = input("Ingrese nombre a buscar: ")
59
60     encontrado = False
61
62     for pais in países:
63
64         if texto.lower() in pais["nombre"].lower():
65
66             print(pais)
67             encontrado = True
68
69     if not encontrado:
70         print("No se encontraron resultados")
71
72 def filtrar_paises(países):
73
74
75     continente = input("Ingrese continente a filtrar: ").strip()
76     poblacion = input("Ingrese rango de población (ej: 1000000-50000000): ").strip()
77     superficie = input("Ingrese rango de superficie (ej: 100000-3000000): ").strip()
78
79     encontrado = False
80
81     for pais in países:
82
83         if pais["continente"].lower() == continente.lower():
84
85             if poblacion:
86                 min_pob, max_pob = map(int, poblacion.split("-"))
87                 if not (min_pob <= pais["poblacion"] <= max_pob):
88                     continue
89
90             if superficie:
91                 min_sup, max_sup = map(int, superficie.split("-"))
92                 if not (min_sup <= pais["superficie"] <= max_sup):
93                     continue
94
95             print(pais)
96             encontrado = True
97
98     if not encontrado:
```

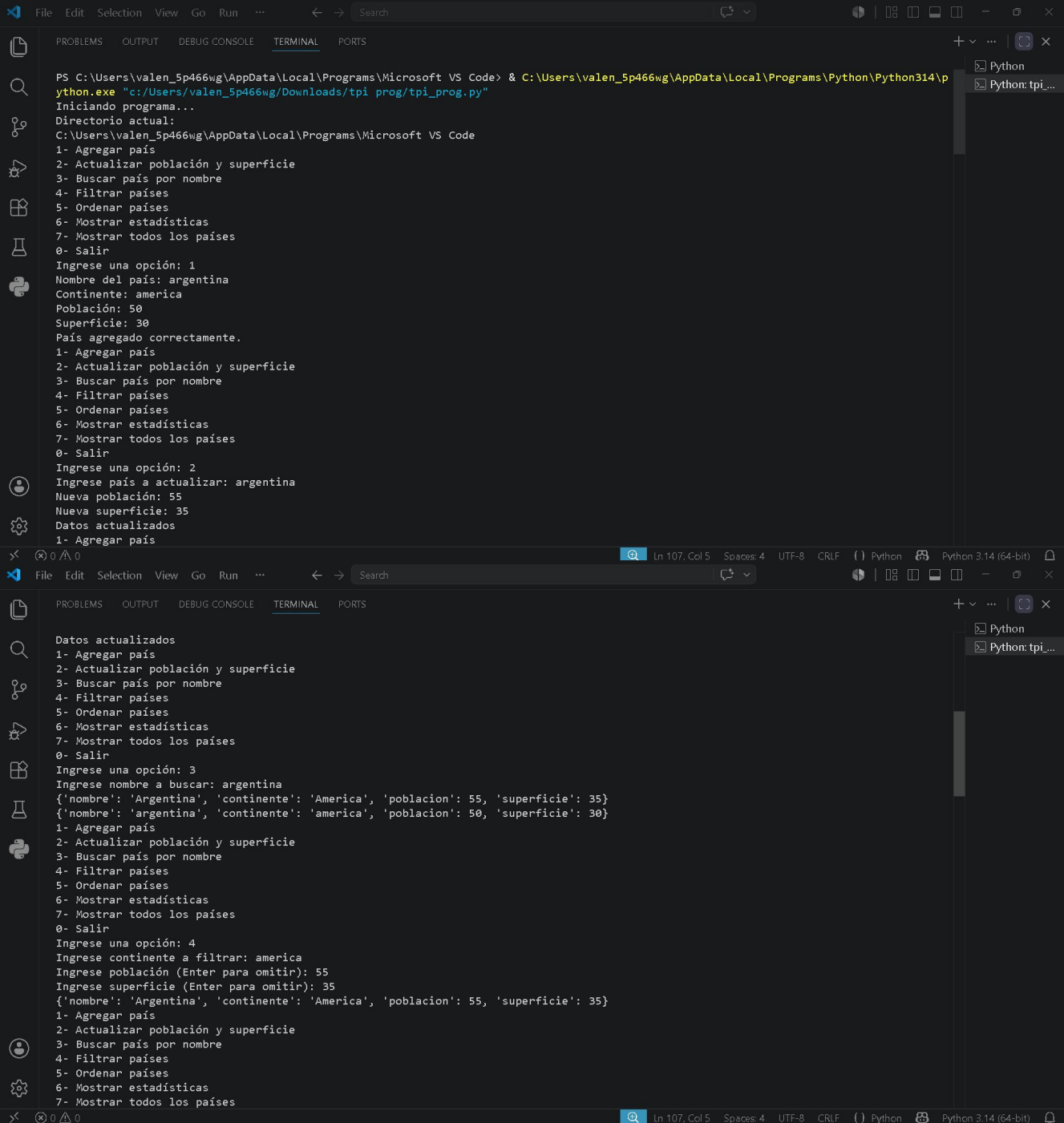
```
File Edit Selection View Go Run ... Search
import csv.py X
C:\Users\valen_5p466wg> Downloads> tpi prog > import csv.py > ...
72 def filtrar_paises(países):
97
98     if not encontrado:
99         print("No se encontraron países con esos filtros")
100 def ordenar_paises(países):
101     criterio = input("Ordenar por (nombre/continente/poblacion/superficie): ").strip()
102
103
104     if criterio in ["nombre", "continente", "poblacion", "superficie"]:
105         ordenados = sorted(países, key=lambda p: p[criterio])
106
107         for pais in ordenados:
108             print(pais)
109     else:
110         print("Criterio no válido")
111
112
113
114 def mostrar_estadisticas(países):
115     mayor = max(países, key=lambda p: p["poblacion"])
116     menor = min(países, key=lambda p: p["poblacion"])
117
118
119     print("Mayor población:", mayor["nombre"], mayor["poblacion"])
120     print("Menor población:", menor["nombre"], menor["poblacion"])
121
122
123
124
125
126
127     for pais in países:
128         suma_pob += pais["poblacion"]
129         suma_sup += pais["superficie"]
130
131
132     prom_pob = suma_pob / len(países)
133     prom_sup = suma_sup / len(países)
134
135
136     print("Promedio población:", prom_pob)
137     print("Promedio superficie:", prom_sup)
138
139
140     continentes = {}
141
142
143     for pais in países:
144
145         cont = pais["continente"]
146
```

```
File Edit Selection View Go Run ... Search
import csv.py X
C: > Users > valen_5p466wg > Downloads > tpi prog > import csv.py > ...
114 def mostrar_estadisticas(paises):
147
148
149     if cont in continentes:
150         continentes[cont] += 1
151     else:
152         continentes[cont] = 1
153
154
155     print("Cantidad por continente:")
156
157
158     for c in continentes:
159         print(c, ":", continentes[c])
160
161 def mostrar_todos(paises):
162     for pais in paises:
163         print(pais)
164
165 while True:
166     print("1- Agregar país")
167     print("2- Actualizar población y superficie")
168     print("3- Buscar país por nombre")
169     print("4- Filtrar países")
170     print("5- Ordenar países")
171     print("6- Mostrar estadísticas")
```

Ln 5, Col 54 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit)

```
import csv.py X
C:\Users\valen_Sp466wg> Downloads> tpi prog > import csv.py > ...
170     print("5- Ordenar países")
171     print("6- Mostrar estadísticas")
172     print("7- Mostrar todos los países")
173     print("0- Salir")
174
175     opcion = input("Ingrese una opción: ")
176
177     if opcion == "1":
178         agregar_pais(países)
179
180     elif opcion == "2":
181         actualizar_pais(países)
182
183     elif opcion == "3":
184         buscar_pais(países)
185
186     elif opcion == "4":
187         filtrar_paises(países)
188
189     elif opcion == "5":
190         ordenar_paises(países)
191
192     elif opcion == "6":
193         mostrar_estadísticas(países)
194
195     elif opcion == "7":
196         mostrar_estadísticas(países)
197
198     elif opcion == "7":
199         mostrar_todos(países)
200
201     elif opcion == "0":
202         print("fin del programa")
203         break
204
205     else:
206         print("Opción no válida")
```

CAPTURAS DEL PROGRAMA EJECUTADO



```
PS C:\Users\valen_5p466wg\AppData\Local\Programs\Microsoft VS Code> & C:\Users\valen_5p466wg\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/valen_5p466wg/Downloads/tpi prog/tpi_prog.py"
Iniciando programa...
Directorio actual:
C:\Users\valen_5p466wg\AppData\Local\Programs\Microsoft VS Code
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 1
Nombre del país: argentina
Continente: america
Población: 50
Superficie: 30
País agregado correctamente.
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 2
Ingrese país a actualizar: argentina
Nueva población: 55
Nueva superficie: 35
Datos actualizados
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 3
Ingrese nombre a buscar: argentina
{'nombre': 'Argentina', 'continente': 'America', 'poblacion': 55, 'superficie': 35}
{'nombre': 'argentina', 'continente': 'america', 'poblacion': 50, 'superficie': 30}
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 4
Ingrese continente a filtrar: america
Ingrese población (Enter para omitir): 55
Ingrese superficie (Enter para omitir): 35
{'nombre': 'Argentina', 'continente': 'America', 'poblacion': 55, 'superficie': 35}
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
```

```
File Edit Selection View Go Run ... Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python
Python: tpi...

0- Salir
Ingrese una opción: 5
1- Nombre
2- Continente
3- Población
4- Superficie
Elegí una opción: 1
{'nombre': 'Argentina', 'continente': 'America', 'poblacion': 55, 'superficie': 35}
{'nombre': 'Brasil', 'continente': 'America', 'poblacion': 214000000, 'superficie': 8516000}
{'nombre': 'España', 'continente': 'Europa', 'poblacion': 47000000, 'superficie': 505000}
{'nombre': 'Japon', 'continente': 'Asia', 'poblacion': 125000000, 'superficie': 378000}
{'nombre': 'Argentina', 'continente': 'america', 'poblacion': 50, 'superficie': 30}
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 5
1- Nombre
2- Continente
3- Población
4- Superficie
Elegí una opción: 2
{'nombre': 'Argentina', 'continente': 'America', 'poblacion': 55, 'superficie': 35}
{'nombre': 'Brasil', 'continente': 'America', 'poblacion': 214000000, 'superficie': 8516000}
{'nombre': 'Japon', 'continente': 'Asia', 'poblacion': 125000000, 'superficie': 378000}
{'nombre': 'España', 'continente': 'Europa', 'poblacion': 47000000, 'superficie': 505000}
{'nombre': 'Argentina', 'continente': 'america', 'poblacion': 50, 'superficie': 30}
1- Agregar país
2- Actualizar población y superficie

Ln 107, Col 5 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit)

File Edit Selection View Go Run ... Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python
Python: tpi...

3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 5
1- Nombre
2- Continente
3- Población
4- Superficie
Elegí una opción: 3
{'nombre': 'argentina', 'continente': 'america', 'poblacion': 50, 'superficie': 30}
{'nombre': 'Argentina', 'continente': 'America', 'poblacion': 55, 'superficie': 35}
{'nombre': 'España', 'continente': 'Europa', 'poblacion': 47000000, 'superficie': 505000}
{'nombre': 'Japon', 'continente': 'Asia', 'poblacion': 125000000, 'superficie': 378000}
{'nombre': 'Brasil', 'continente': 'America', 'poblacion': 214000000, 'superficie': 8516000}
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 5
1- Nombre
2- Continente
3- Población
4- Superficie
Elegí una opción: 4
{'nombre': 'argentina', 'continente': 'america', 'poblacion': 50, 'superficie': 30}
{'nombre': 'Argentina', 'continente': 'America', 'poblacion': 55, 'superficie': 35}

Ln 107, Col 5 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit)
```



```
File Edit Selection View Go Run ... Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python
Python: tpi...

{'nombre': 'Argentina', 'continente': 'America', 'poblacion': 55, 'superficie': 35}
{'nombre': 'Japon', 'continente': 'Asia', 'poblacion': 125000000, 'superficie': 378000}
{'nombre': 'España', 'continente': 'Europa', 'poblacion': 47000000, 'superficie': 505000}
{'nombre': 'Brasil', 'continente': 'America', 'poblacion': 214000000, 'superficie': 8516000}
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 6
Mayor población: Brasil 214000000
Menor población: argentina 50
Promedio población: 77200021.0
Promedio superficie: 1879813.0
Cantidad por continente:
America : 2
Europa : 1
Asia : 1
america : 1
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 7
{'nombre': 'Argentina', 'continente': 'America', 'poblacion': 55, 'superficie': 35}
{'nombre': 'Brasil', 'continente': 'America', 'poblacion': 214000000, 'superficie': 8516000}

Ln 107, Col 5 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit)

File Edit Selection View Go Run ... Search
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python
Python: tpi...

2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 7
{'nombre': 'Argentina', 'continente': 'America', 'poblacion': 55, 'superficie': 35}
{'nombre': 'Brasil', 'continente': 'America', 'poblacion': 214000000, 'superficie': 8516000}
{'nombre': 'España', 'continente': 'Europa', 'poblacion': 47000000, 'superficie': 505000}
{'nombre': 'Japon', 'continente': 'Asia', 'poblacion': 125000000, 'superficie': 378000}
{'nombre': 'argentina', 'continente': 'america', 'poblacion': 50, 'superficie': 30}
1- Agregar país
2- Actualizar población y superficie
3- Buscar país por nombre
4- Filtrar países
5- Ordenar países
6- Mostrar estadísticas
7- Mostrar todos los países
0- Salir
Ingrese una opción: 0
fin del programa
PS C:\Users\valen_5p466wg\AppData\Local\Programs\Microsoft VS Code>
```


Dificultades y soluciones

La mayor dificultad surgió en el momento de crear el código, pero se solucionó buscando en la teoría o mirando videos sobre las dudas que surgían mientras íbamos creando respectivo código

Respecto a los temas que nos surgieron dudas fueron:

- funciones
- archivos CSV
- manejo de errores
- estructuración del código

Estos temas fueron los principales que nos generaron dificultades pero con ayudas de la teoría, videos, se pudo lograr llevar adelante

Conclusion

La realización de este Trabajo Práctico Integrador me permitió poner en práctica los conceptos vistos durante la cursada de Programación 1. A través del desarrollo del sistema de gestión de países, pude trabajar con listas, diccionarios, funciones, estructuras condicionales y repetitivas, además de aprender a utilizar archivos CSV para almacenar y leer información.

Durante el desarrollo surgieron algunas dificultades, principalmente relacionadas con la organización del código, la implementación de funciones y el manejo de errores. Sin embargo, a medida que fuimos resolviendo estos problemas, logramos comprender mejor el funcionamiento del lenguaje Python y la importancia de escribir programas ordenados y fáciles de mantener.

Consideramos que este proyecto fue una experiencia muy útil, ya que permitió integrar varios de los temas trabajados en la materia dentro de una aplicación funcional. Además, ayudó a desarrollar habilidades de análisis, resolución de problemas y búsqueda de soluciones, que son fundamentales para la formación como programador.

En conclusión, este trabajo no solo sirvió para cumplir con los objetivos de la materia, sino también para reforzar conocimientos y adquirir mayor confianza al momento de desarrollar programas utilizando Python.